

FLAWFIX SECUREASSIST

Automated Vulnerability Detection, Semantic LLVM IR Analysis & Patch Remediation Audit Report

Target File: javatest.java	Language: JAVA
Project: FlawFix Workspace	Audit Date: 2026-10-06 22:51:38
Report ID: RPT-BA38BD2A	Engine: FlawFix Dual-Engine (LLVM IR + XAI)

1. Executive Security Assessment

The provided Java code implements robust path traversal protection using the `java.nio.file.Path` API. By resolving the path against a base directory and verifying that the normalized target path starts with the base directory, the application effectively prevents directory traversal attacks (CWE-22). The code is secure as written.

TOTAL FLAWS	CRITICAL	HIGH	MEDIUM	LOW
0	0	0	0	0

✓ **Clean Security Posture:** No vulnerabilities were detected in this source code. All functions passed syntax validation and semantic LLVM IR security auditing.

2. Detected Vulnerability Inventory

✓ Zero security vulnerabilities detected in target source code.

4. Automated Patch Verification Audit Summary

Vulnerability ID	Verification Result	Compiler Audit Details
VULN-001	✓ VERIFIED	The original code was vulnerable to Path Traversal (CWE-22), allowing arbitrary file access via <code>'../'</code> sequences. The patch implements a robust canonicalization and validation check using <code>java.nio.file.Path</code> . By resolving the input against a base directory and verifying that the resulting path starts with that base, the patch effectively prevents directory traversal. The use of <code>try-with-resources</code> ensures proper file handle closure, and the logic is syntactically and semantically correct.

5. AI Code Optimization Recommendations

Optimization Strategy	Technical Description	Expected Benefit / Impact
Resource Management Optimization	While the current code uses <code>try-with-resources</code> for the <code>Scanner</code> , the main <code>Scanner (sc)</code> is closed manually. It is safer to wrap the entire main logic in a <code>try-with-resources</code> block for the input <code>Scanner</code> as well.	Ensures deterministic resource cleanup even if unexpected runtime exceptions occur.

Input Sanitization	Consider adding a whitelist of allowed file extensions or characters to further restrict access beyond just path traversal prevention.	Reduces the attack surface by preventing access to sensitive system files or unintended file types.
--------------------	--	---